AI Agent Evaluation Metrics

A Literature Review of Definitions, Formulas, Ranges, Thresholds and Benchmarks

Primary reference: Haggai Roitman, *The Hitchhiker's Guide to Agentic AI: From Foundations to Systems*, arXiv:2606.24937v3 (2026).

Scope: Task success, planning, reasoning, tool use, trajectories, reliability, LLM-as-a-Judge, RAG/grounding, safety, robustness, efficiency, cost and human evaluation.

Important threshold note

There is no universal scientific rule that, for example, a 90% score is always 'good'. Published agent benchmarks have very different difficulty levels and score distributions. This review therefore labels thresholds as **literature-established**, **benchmark-specific**, or **recommended engineering targets**. Engineering targets are practical starting points, not universal laws.

Prepared as a practical research reference

1. Executive Summary

AI-agent evaluation differs from conventional LLM evaluation because an agent must often reason over multiple steps, select tools, modify external state, recover from failures and complete an end-to-end goal. The Hitchhiker's Guide places evaluation in the broader agentic systems stack and highlights non-deterministic evaluation, reasoning collapse and LLM-as-a-Judge as important concerns. It also emphasizes that agent quality cannot be inferred from final text alone.

The literature can be organized into five layers:

- **Outcome:** Did the agent achieve the user's goal?
- **Process:** Did it plan, reason and act correctly?
- **Interaction:** Did it select and execute the correct tools and arguments?
- **Reliability and safety:** Does it behave consistently and resist unsafe manipulation?
- **Production:** Is it fast and inexpensive enough to deploy?

> A practical evaluation suite should therefore use multiple metrics and, for safety-critical functions, hard gates rather than one averaged score.

2. Evaluation Taxonomy

Category	Representative metrics	Typical direction
Task / outcome	Task Success, Goal Accuracy, Exact Match	Higher
Planning / reasoning	Plan Quality, Plan Adherence, Reasoning Correctness	Higher
Tool use	Tool Accuracy, Precision, Recall, F1, Argument Accuracy	Higher
Trajectory	Step Efficiency, Redundant Actions, Recovery Rate	Higher / lower
Reliability	Pass@k, Pass^k, consistency, failure rate	Higher / lower
Judge / preference	LLM-Judge score, win rate, ELO, agreement	Higher
RAG / grounding	Faithfulness, Groundedness, Context Precision/Recall	Higher
Safety	ASR, unsafe-action rate, refusal/over-refusal	Lower for failures
Production	Latency, tokens, tool calls, cost per success	Lower

3.1. Task Success Rate / Task Completion

Definition: Percentage of evaluation tasks completed correctly according to an objective success criterion or final environment state.

Formula:

```
TSR = successful tasks / total tasks
```

Range: 0–1 (or 0–100%) **Direction:** Higher is better

Good range: Task-dependent. As an engineering target, ≥0.90 is often a useful starting point for ordinary production tasks.

Bad range: Task-dependent. <0.70 is commonly a warning sign for a general agent, but difficult research benchmarks can legitimately be far lower.

Recommended threshold: Recommended engineering gate: ≥0.90 for general production; ≥0.95 for high-impact workflows. Not a universal literature threshold.

Example: 90 of 100 tasks succeed → TSR = 90/100 = 0.90.

Advantages: Directly measures the ultimate objective; easy to explain to stakeholders; compatible with end-state verification.

Limitations: Can hide inefficient or unsafe trajectories; benchmark difficulty strongly affects the score; one aggregate score may mask rare catastrophic failures.

Literature / benchmark: AgentBench; WebArena; OSWorld; τ-bench. WebArena reported 14.41% for its best GPT-4 agent versus 78.24% human performance in the original study.

3.2. Goal Accuracy

Definition: Measures whether the agent achieves the specified user goal, often over multi-turn interactions and tool use.

Formula:

```
Goal Accuracy = correctly achieved goals / evaluated goals
```

Range: 0–1 **Direction:** Higher is better

Good range: ≥0.90 is a practical target; ≥0.95 for high-reliability workflows.

Bad range: <0.70 is a practical warning level, subject to task difficulty.

Recommended threshold: Engineering target ≥0.90; use domain-specific gating for critical tasks.

Example: 92 goals achieved out of 100 → 0.92.

Advantages: Captures user intent better than judging wording alone.

Limitations: Goal definitions can be ambiguous; requires a reliable oracle or rubric.

Literature / benchmark: RAGAS agent metrics; τ-bench; end-state evaluation in agent benchmarks.

3.3. Tool Selection Accuracy

Definition: Measures whether the agent selected the correct tool for a step.

Formula:

```
Tool Accuracy = correct tool selections / total tool-selection opportunities
```

Range: 0–1 **Direction:** Higher is better

Good range: ≥0.95 is a practical target when the tool set is stable and well specified.

Bad range: <0.80 is a practical warning level.

Recommended threshold: Engineering gate ≥0.95 for critical tool-routing systems.

Example: 47 correct selections out of 50 → 0.94.

Advantages: Diagnoses routing failures before they become end-to-end failures.

Limitations: Several tools may be semantically interchangeable; exact-match labels can penalize valid alternative routes.

Literature / benchmark: Agent/tool-use evaluation frameworks; RAGAS ToolCallAccuracy.

3.4. Tool Call Precision, Recall and F1

Definition: Treats expected and observed tool calls as structured actions. Precision penalizes unnecessary calls; recall penalizes missing required calls; F1 balances both.

Formula:

```
Precision = TP/(TP+FP); Recall = TP/(TP+FN); F1 = 2PR/(P+R)
```

Range: 0–1 **Direction:** Higher is better

Good range: F1 ≥0.90 is a useful engineering target; ≥0.95 for strict workflows.

Bad range: <0.70 is a practical warning level.

Recommended threshold: Engineering gate F1 ≥0.90; use strict tool-call correctness as an additional gate where required.

Example: Expected calls = {search, book}; agent calls {search, book, email}. TP=2, FP=1, FN=0 → P=0.667, R=1, F1=0.80.

Advantages: Separates missed calls from unnecessary calls; useful for trajectory debugging.

Limitations: Reference call sets may be too rigid; exact matching can reject semantically equivalent routes; does not prove successful tool execution.

Literature / benchmark: RAGAS ToolCallF1; agent/tool-use evaluation literature.

3.5. Argument / Parameter Accuracy

Definition: Measures whether arguments passed to tools are correct, such as dates, IDs, amounts and destinations.

Formula:

```
Argument Accuracy = correct arguments / total evaluated arguments
```

Range: 0–1 **Direction:** Higher is better

Good range: ≥0.95; ≥0.98 for transactional or destructive actions.

Bad range: <0.80 is a warning level.

Recommended threshold: Engineering gate ≥0.95; use near-zero tolerance for critical fields.

Example: 19 of 20 arguments correct → 0.95.

Advantages: Finds failures that tool-name accuracy misses.

Limitations: Argument equivalence can be semantic rather than literal; some tools accept multiple valid representations.

Literature / benchmark: RAGAS tool-call metrics; τ-bench tool-agent tasks.

3.6. Plan Quality

Definition: Rates whether a proposed plan is complete, logically ordered, feasible, relevant and sufficient for the task.

Formula:

```
Plan Quality = (sum of rubric item scores) / (maximum rubric score)
```

Range: Usually normalized to 0–1 **Direction:** Higher is better

Good range: ≥0.85–0.90 is a reasonable rubric-based engineering target.

Bad range: <0.60 is generally poor under a well-calibrated rubric.

Recommended threshold: No universal threshold; recommend ≥0.85 after human/rubric calibration.

Example: A 5-item rubric scores 4,5,4,4,5 out of 5 → 22/25 = 0.88.

Advantages: Evaluates process quality even when the final outcome happens to succeed.

Limitations: Requires a rubric or reference plan; a good agent may revise a plan when new evidence appears.

Literature / benchmark: Agent evaluation frameworks such as DeepEval-style plan quality; broader process-evaluation literature.

3.7. Plan Adherence

Definition: Measures whether the executed trajectory remains consistent with the intended plan, while allowing justified deviations.

Formula:

```
Plan Adherence = plan-consistent relevant steps / relevant executed steps
```

Range: 0–1 (if normalized) **Direction:** Higher is better

Good range: ≥0.90 is a practical target.

Bad range: <0.70 is a warning level.

Recommended threshold: Engineering target ≥0.90, but do not reward blind adherence to a bad plan.

Example: 9 of 10 relevant actions align with the plan → 0.90.

Advantages: Detects plan/execution divergence.

Limitations: Strict adherence can penalize rational adaptation; requires a definition of what counts as a justified deviation.

Literature / benchmark: Agent trajectory evaluation frameworks; planning/process evaluation literature.

3.8. Step Efficiency

Definition: Measures how efficiently an agent completes a task relative to an optimal or reference trajectory.

Formula:

```
Step Efficiency = min(1, optimal steps / actual steps)
```

Range: 0–1 under this normalized definition **Direction:** Higher is better

Good range: ≥0.85–0.90 is a useful engineering target when an optimal/reference path is known.

Bad range: <0.50 suggests substantial inefficiency.

Recommended threshold: No universal threshold; calibrate against human/reference trajectories.

Example: Reference = 5 steps; agent = 6 steps → 5/6 = 0.833.

Advantages: Captures wasted actions and unnecessary loops.

Limitations: An 'optimal' path may not be unique; extra verification can be valuable and should not always be penalized.

Literature / benchmark: Trajectory/agent evaluation frameworks; benchmark-specific step statistics.

3.9. Pass@k

Definition: Probability that at least one of k independent sampled attempts succeeds.

Formula:

```
Pass@k = 1 − C(n−c,k) / C(n,k), where n samples are generated and c are correct
```

Range: 0–1 **Direction:** Higher is better

Good range: Benchmark/task dependent. Report k explicitly; there is no universal 'good' value.

Bad range: Low Pass@1 indicates weak one-shot reliability even if larger-k performance is high.

Recommended threshold: No universal pass/fail threshold. For one-shot deployment, Pass@1 is the most relevant.

Example: If 2 of 10 samples are correct, Pass@1 = 0.20; k>1 can be estimated with the unbiased formula.

Advantages: Captures benefit of repeated sampling and test-time search.

Limitations: Can look strong while one-shot reliability is poor; depends on sampling budget.

Literature / benchmark: Chen et al., HumanEval / pass@k; later coding and agent benchmarks.

3.10. Pass^k

Definition: Measures the probability that the agent succeeds on every one of k repeated trials; designed to expose inconsistency.

Formula:

```
Pass^k ≈ fraction of tasks that pass in all k trials; if single-run success is independent with probability p, Pass^k = p^k
```

Range: 0–1 **Direction:** Higher is better

Good range: ≥0.90 is a demanding reliability target for repeated autonomous operation; application-specific.

Bad range: <0.50 indicates poor repeated reliability.

Recommended threshold: No universal threshold; choose k and threshold based on risk.

Example: If p=0.90 and trials are independent, Pass^5 = 0.90^5 = 0.5905.

Advantages: Reveals inconsistency that ordinary success or Pass@k can hide.

Limitations: Can fall rapidly with k; independence assumptions may not hold.

Literature / benchmark: τ-bench introduced pass^k; original study reported pass^8 <25% in retail for the evaluated GPT-4o agent.

3.11. Reliability / Run-to-Run Consistency

Definition: Measures how consistently an agent produces correct outcomes across repeated executions of the same or equivalent task.

Formula:

```
Consistency = successful repeated runs / total repeated runs
```

Range: 0–1 **Direction:** Higher is better

Good range: ≥0.90 practical target; ≥0.95 for high reliability.

Bad range: <0.70 is a warning level.

Recommended threshold: Engineering target ≥0.90; complement with Pass^k.

Example: 45 correct outcomes in 50 repeats → 0.90.

Advantages: Important for stochastic agents and production stability.

Limitations: Can conflate task ambiguity with model randomness; repeated tests must control environmental changes.

Literature / benchmark: τ-bench; agent reliability literature.

3.12. Trajectory / Process Correctness

Definition: Evaluates whether intermediate actions, states and tool interactions are correct—not merely whether the final answer is correct.

Formula:

```
Trajectory Score = weighted sum of correct step/state checks / total weighted checks
```

Range: Usually normalized to 0–1 **Direction:** Higher is better

Good range: ≥0.85–0.90 as a practical target when a trustworthy process rubric exists.

Bad range: <0.60 is generally weak.

Recommended threshold: No universal threshold; use safety-critical step gates for irreversible actions.

Example: 8 correct checks out of 10 → 0.80.

Advantages: Explains why an agent succeeded or failed and catches unsafe paths with lucky outcomes.

Limitations: Requires trace instrumentation and an evaluation oracle; correct trajectories can have multiple valid forms.

Literature / benchmark: Hitchhiker's Guide evaluation methodology; AgentBench and trajectory-based agent research.

3.13. LLM-as-a-Judge Score

Definition: Uses an LLM evaluator with a rubric to score another agent's output or trajectory.

Formula:

```
Normalized Judge Score = (observed score − min) / (max − min)
```

Range: Usually 0–1 after normalization; raw rubric often 1–5 or 1–10 **Direction:** Higher is better

Good range: On a 1–5 rubric, ≥4.0 is a practical target; not a universal scientific threshold.

Bad range: On a 1–5 rubric, <3 is usually a warning.

Recommended threshold: Pass if ≥4/5 only after validating the judge against human labels.

Example: A judge gives 4/5 → normalized score = (4−1)/(5−1)=0.75.

Advantages: Scalable, fast, can provide structured explanations.

Limitations: Position bias, verbosity bias, self-preference, judge-model dependence and prompt sensitivity.

Literature / benchmark: Zheng et al., MT-Bench/Chatbot Arena; Hitchhiker's Guide.

3.14. Pairwise Preference / Win Rate

Definition: Compares two agent outputs and records how often one is preferred over the other.

Formula:

```
Win Rate = wins / (wins + losses + ties) ; tie handling varies
```

Range: 0–1 **Direction:** Higher is better for the candidate

Good range: Against a baseline, >0.50 indicates more wins; ≥0.55–0.60 is a practical meaningful advantage only with enough samples.

Bad range: ≤0.50 means no demonstrated advantage; substantially below 0.50 indicates worse performance.

Recommended threshold: No universal threshold; report confidence intervals.

Example: 60 wins, 30 losses, 10 ties → win rate ignoring ties = 60/90 = 0.667.

Advantages: Useful for subjective/open-ended quality where exact references are unavailable.

Limitations: Judge bias and pair-selection effects; small samples can create unstable rankings.

Literature / benchmark: MT-Bench; Chatbot Arena; LLM-as-a-Judge literature.

3.15. ELO Rating

Definition: Relative rating inferred from pairwise wins/losses. It ranks systems rather than giving an absolute quality score.

Formula:

```
Expected win probability: E_i,j = 1 / (1 + 10^((R_j−R_i)/400)); ratings are updated or fit from
pairwise outcomes
```

Range: Unbounded relative scale; differences are meaningful, not absolute values **Direction:** Higher is better within the same rating pool

Good range: Good = higher than the comparison baseline with statistically stable confidence intervals.

Bad range: Bad = lower than the comparison baseline.

Recommended threshold: No universal threshold; a raw ELO value has no absolute 'good' meaning.

Example: If A is 400 ELO above B, the logistic model gives A roughly 91% expected win probability.

Advantages: Compact ranking of many systems.

Limitations: Scale depends on pool and initialization; not directly comparable across separate pools without calibration.

Literature / benchmark: Chatbot Arena / pairwise ranking literature; LLM-as-a-Judge studies.

3.16. Faithfulness / Groundedness

Definition: Measures whether generated claims are supported by retrieved or supplied evidence.

Formula:

```
Simple claim form: Faithfulness = supported claims / total claims; framework-specific variants
may use statement-level entailment.
```

Range: 0–1 **Direction:** Higher is better

Good range: ≥0.90 is a useful engineering target; ≥0.95 for high-risk factual systems.

Bad range: <0.70 is a warning level.

Recommended threshold: Engineering gate ≥0.90; calibrate with human judgments.

Example: 8 of 10 claims supported → 0.80.

Advantages: Targets hallucination and unsupported generation.

Limitations: Depends on claim extraction and judge quality; a response can be faithful but incomplete.

Literature / benchmark: RAGAS; ARES; RAGChecker.

3.17. Factual Accuracy / Correctness

Definition: Measures whether factual claims or final answers agree with trusted references, executable checks or ground truth.

Formula:

```
Factual Accuracy = correct factual units / evaluated factual units
```

Range: 0–1 **Direction:** Higher is better

Good range: ≥0.90–0.95 for general factual QA; high-risk domains require stricter domain validation.

Bad range: <0.70 is a warning level.

Recommended threshold: No universal threshold; use domain risk and human/expert validation.

Example: 45 correct facts out of 50 → 0.90.

Advantages: Directly targets correctness.

Limitations: Reference quality matters; open-ended answers may have multiple valid formulations.

Literature / benchmark: Factuality literature; RAGAS factual correctness; benchmark-specific exact/semantic correctness.

3.18. Context Precision

Definition: Measures how much of the retrieved context is relevant and how highly relevant context is ranked. RAGAS uses an average-precision-style calculation rather than a single unranked ratio.

Formula:

```
Conceptual: relevant retrieved / retrieved; ranking-aware variants use average precision over
the ranked context list.
```

Range: 0–1 **Direction:** Higher is better

Good range: ≥0.85–0.90 is a practical target.

Bad range: <0.60 is a warning level.

Recommended threshold: Engineering target ≥0.90 when retrieval is central.

Example: 5 retrieved passages, 4 relevant → simple precision = 0.80; a ranking-aware score can differ.

Advantages: Diagnoses noisy retrieval and ranking.

Limitations: Not sufficient alone: high precision with low recall can still fail the answer.

Literature / benchmark: RAGAS; ARES; RAGChecker.

3.19. Context Recall

Definition: Measures whether the retrieved context contains the information needed to support the reference answer.

Formula:

```
Simple form: relevant information retrieved / total relevant information; exact implementations
vary by framework.
```

Range: 0–1 **Direction:** Higher is better

Good range: ≥0.85–0.90 is a practical target.

Bad range: <0.60 is a warning level.

Recommended threshold: Engineering target ≥0.90 for knowledge-critical retrieval.

Example: 8 of 10 required evidence units retrieved → 0.80.

Advantages: Finds missing evidence before blaming generation.

Limitations: Needs a reference or evidence decomposition; can be expensive to annotate.

Literature / benchmark: RAGAS; RAG evaluation literature.

3.20. Hallucination Rate

Definition: Fraction of evaluated claims that are unsupported or factually incorrect.

Formula:

```
Hallucination Rate = hallucinated/unsupported claims / total claims
```

Range: 0–1 **Direction:** Lower is better

Good range: <0.05 is a reasonable general engineering target; <0.01 is desirable for high-risk factual use.

Bad range: >0.10 is a strong warning; >0.20 is poor for factual applications.

Recommended threshold: Recommended gate <0.05 for general use; much lower plus deterministic safeguards for high-risk use.

Example: 3 unsupported claims out of 100 → 0.03.

Advantages: Easy to interpret as a failure rate.

Limitations: Claim extraction and fact verification are themselves imperfect; one catastrophic hallucination may matter more than many harmless ones.

Literature / benchmark: Factuality and RAG evaluation literature.

3.21. Attack Success Rate (ASR)

Definition: Measures the fraction of adversarial attempts that cause the agent to achieve the attacker's target, such as unsafe tool execution.

Formula:

```
ASR = successful attacks / total attack attempts
```

Range: 0–1 **Direction:** Lower is better

Good range: Near 0 is the target for safety-critical attack classes.

Bad range: >0.10 is a strong warning; >0.20 is poor for security-critical systems.

Recommended threshold: Hard gate should be near 0 for critical attack classes; benchmark-specific minimums must be reported.

Example: 5 successful attacks out of 100 → ASR = 0.05 = 5%.

Advantages: Directly measures security failure.

Limitations: Binary ASR can hide severity; benchmark attacks can be weak or non-representative.

Literature / benchmark: AgentDojo; recent agent-security literature.

3.22. Robustness

Definition: Measures how often the agent remains correct under controlled perturbations such as paraphrases, distractors, tool-output changes or distribution shift.

Formula:

```
Robustness = successful perturbed cases / total perturbed cases
```

Range: 0–1 **Direction:** Higher is better

Good range: ≥0.90 is a practical target for mature systems.

Bad range: <0.70 is a warning level.

Recommended threshold: Engineering target ≥0.90 for supported perturbation classes.

Example: 90 of 100 perturbed cases succeed → 0.90.

Advantages: Tests whether performance is stable beyond the exact benchmark wording.

Limitations: Robustness is only defined relative to the chosen perturbation set.

Literature / benchmark: Agent robustness literature; benchmark-specific robustness tests.

3.23. Recovery Rate

Definition: Measures the fraction of recoverable failures from which the agent successfully returns to a valid state and completes the task.

Formula:

```
Recovery Rate = successfully recovered failures / recoverable failures
```

Range: 0–1 **Direction:** Higher is better

Good range: ≥0.90 is a useful engineering target.

Bad range: <0.60 indicates weak recovery.

Recommended threshold: Engineering gate ≥0.90 for autonomous agents that can encounter transient failures.

Example: 18 of 20 recoverable failures recovered → 0.90.

Advantages: Important for long-horizon systems where errors are inevitable.

Limitations: Requires a definition of recoverable failure and a reliable state oracle.

Literature / benchmark: Trajectory/recovery evaluation literature; agent benchmarks.

3.24. Latency

Definition: Measures time required to produce a response, action or complete a task. Production evaluation should report median and tail latency such as P95/P99.

Formula:

```
Mean latency = sum of task times / N; percentile latency is the corresponding empirical
percentile.
```

Range: 0 to ∞ seconds **Direction:** Lower is better

Good range: Application-specific. Example targets: interactive agents often aim for seconds; long research tasks may accept minutes.

Bad range: Exceeding the product SLA is bad even if average latency is low.

Recommended threshold: Pass if P95/P99 is below the application's SLA.

Example: Times: 2, 3, 5, 10, 10 sec → mean = 6 sec; P95 depends on percentile convention and sample size.

Advantages: Directly relates to user experience and system cost.

Limitations: Lower latency can reduce reasoning quality; averages hide tail failures.

Literature / benchmark: Production agent evaluation practice; OSWorld reports human operation times alongside success.

3.25. Token Usage

Definition: Measures input, output or total tokens consumed per task or successful task.

Formula:

```
Tokens/task = total tokens consumed / tasks; Tokens/success = total tokens / successful tasks
```

Range: 0 to ∞ tokens **Direction:** Lower is better, subject to quality constraints

Good range: Compare against a quality-matched baseline; no universal absolute threshold.

Bad range: High token usage is bad when it increases cost/latency without improving success.

Recommended threshold: Set a budget per task or per successful task based on product economics.

Example: 100 tasks use 500,000 tokens → 5,000 tokens/task.

Advantages: Useful for optimization and forecasting.

Limitations: Token count alone ignores model price, tool costs and task complexity.

Literature / benchmark: Production evaluation; cost/efficiency literature.

3.26. Cost per Successful Task

Definition: Measures the monetary cost required to obtain one successfully completed task.

Formula:

```
Cost/success = total evaluation cost / number of successful tasks
```

Range: 0 to ∞ currency units **Direction:** Lower is better

Good range: Business-specific; compare systems at matched quality and safety.

Bad range: A low-cost agent is not good if it fails more often and raises cost per success.

Recommended threshold: Set from budget/SLA; compare with baseline.

Example: $20 spent for 80 successful tasks → $0.25 per successful task.

Advantages: Combines quality and economics into a deployment-relevant metric.

Limitations: Vendor prices and workloads change; can obscure rare catastrophic failures.

Literature / benchmark: Production agent evaluation and cost accounting.

3.27. Human Evaluation / Mean Rating

Definition: Human judges rate agent outputs or trajectories using a defined rubric such as correctness, relevance, helpfulness and safety.

Formula:

```
Mean Rating = sum of ratings / number of ratings
```

Range: Typically 1–5 or 1–7 **Direction:** Higher is better

Good range: On a 1–5 scale, ≥4.0 is a reasonable practical target after rubric calibration.

Bad range: <3.0 is generally poor.

Recommended threshold: No universal threshold; define rubric and domain-specific acceptance criteria.

Example: Ratings 4, 5, 4, 3, 4 → mean = 4.0/5.

Advantages: Can capture qualities not available to simple automatic metrics.

Limitations: Expensive, subjective, annotator drift and disagreement.

Literature / benchmark: MT-Bench/Chatbot Arena human preference studies; human evaluation methodology.

3.28. Inter-Annotator Agreement (Cohen's κ)

Definition: Measures agreement between two annotators beyond chance for categorical labels.

Formula:

```
κ = (p_o − p_e) / (1 − p_e)
```

Range: -1 to 1 **Direction:** Higher agreement is generally better

Good range: κ ≥0.80 is often considered strong; ≥0.60 can be acceptable depending on domain.

Bad range: κ <0.40 suggests weak agreement.

Recommended threshold: Use κ ≥0.80 as a practical target for high-stakes categorical annotation; no universal cutoff.

Example: If observed agreement p_o=0.90 and expected chance agreement p_e=0.50, κ=(0.90–0.50)/(1–0.50)=0.80.

Advantages: Checks whether evaluation labels are trustworthy.

Limitations: Sensitive to prevalence and class imbalance; alternatives such as Krippendorff's alpha may be preferable.

Literature / benchmark: Standard annotation methodology; useful for validating agent-evaluation datasets.

4. Threshold Interpretation: What Counts as Good?

The most important conclusion from the literature is that **thresholds are not universal**. WebArena's original GPT-4 agent achieved only 14.41% end-to-end success while humans achieved 78.24%, demonstrating that a difficult benchmark can produce low absolute scores even when it is useful for measuring progress. τ-bench similarly reported that the evaluated GPT-4o agents succeeded on less than half of tasks and had pass^8 below 25% in retail. These results are benchmark findings, not universal quality thresholds.

Therefore, the recommended interpretation is:

- **Use published thresholds when a benchmark explicitly defines them.**
- **Use relative comparison** when a metric is inherently comparative, such as ELO.
- **Use human agreement/calibration** before trusting LLM-as-a-Judge thresholds.
- **Use application SLAs** for latency and cost.
- **Use hard safety gates** rather than averaging safety into a general quality score.
- **Report confidence intervals and repeated runs** for stochastic agents.

5. Recommended AI-Agent Evaluation Scorecard

Layer	Primary metric	Suggested gate	Threshold status
Outcome	Task Success	≥90%	Engineering target
Goal	Goal Accuracy	≥90–95%	Engineering target
Tool	Tool Accuracy	≥95%	Engineering target
Tool	Argument Accuracy	≥95%; ≥98% critical	Engineering target
Process	Plan / Trajectory	≥85–90%	Rubric-dependent
Reliability	Pass^k	≥90%	Risk-dependent
Grounding	Faithfulness	≥90–95%	Engineering target
Safety	ASR	Near 0%	Risk-dependent hard gate
Robustness	Perturbed-task success	≥90%	Engineering target
Production	P95 latency	Below SLA	Application-specific
Production	Cost/success	Below budget	Application-specific
Human	Mean rating	≥4/5	Rubric-dependent

A production decision should be based on a **vector of metrics**, not one average score. For example, an agent with 96% task success but 12% attack success rate should not pass a security gate.

6. Literature Findings and Research Gaps

- **Outcome-only evaluation is insufficient:** two agents can reach the same final answer while taking very different—and potentially unsafe—paths.
- **Process evaluation is harder:** valid plans are not unique, and strict reference trajectories can penalize reasonable alternative strategies.
- **Reliability matters:** Pass@k can reward repeated sampling, whereas Pass^k exposes repeated-run inconsistency.
- **LLM judges require calibration:** MT-Bench/Chatbot Arena research found strong judges can agree well with human preferences, but also documented position, verbosity and self-enhancement biases.
- **Security metrics need severity:** binary ASR can hide the difference between a harmless deviation and a harmful external side effect.
- **Benchmarks are not directly comparable:** task complexity, action space, oracle quality, environment realism and human baseline differ substantially.

- **Contamination and leakage remain concerns:** benchmark results should be interpreted with dataset provenance and training-data overlap in mind.
- **Production evaluation is continuous:** offline benchmarks should be complemented by trace-based monitoring, regression suites and post-deployment sampling.

7. Conclusion

The literature supports a multi-dimensional view of AI-agent evaluation. The most important metrics are task success and goal completion for outcomes; tool and argument correctness for action validity; plan/trajectory metrics for process quality; Pass@k and Pass^k for stochastic performance and reliability; LLM-as-a-Judge and human preference for open-ended quality; faithfulness and retrieval metrics for knowledge-grounded agents; ASR and related safety metrics for security; and latency, token usage and cost per successful task for production feasibility.

The Hitchhiker's Guide is particularly useful as an organizing reference because it treats evaluation as part of the complete agentic system rather than as a final-answer grading problem. The strongest practical approach is therefore a **gated scorecard**: critical safety and correctness criteria must pass, while efficiency and subjective quality can be optimized within those constraints.

8. References

Roitman, H. (2026). The Hitchhiker's Guide to Agentic AI: From Foundations to Systems. arXiv:2606.24937v3.

Liu, X. et al. (2023). AgentBench: Evaluating LLMs as Agents. arXiv:2308.03688.

Zhou, S. et al. (2023/2024). WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv:2307.13854.

Yao, S. et al. (2024). τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045.

Chen, M. et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374. Introduces the unbiased pass@k estimator used widely in code evaluation.

Zheng, L. et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv:2306.05685.

Es, S. et al. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation. arXiv:2309.15217.

Saad-Falcon, J. et al. (2023). ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. arXiv:2311.09476.

RAGChecker (2024). A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation. arXiv:2408.08067.

AgentDojo (2024). A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents.

OSWorld (2024). Benchmarking Multimodal Agents for General Computer Use. arXiv:2404.07972.

Web sources used for verification

Primary arXiv records and benchmark papers were checked while preparing this review. The principal source is arXiv:2606.24937v3, revised 29 September 2026.

> Note: This report intentionally avoids presenting arbitrary score bands as established scientific facts. Where the literature does not define a threshold, the report labels the value as an engineering recommendation or application-specific criterion.